

LeafGuard AI — ML Project Deployment Phase 1

Milestone 4: Folder Structure, Modular Backend & Streamlit Interface | Lead: Tayyab Sohail

1. Technology Stack Selection

The deployment framework for LeafGuard AI utilizes a modular, Python-centric architecture:

- **Web Frontend UI:** Streamlit 1.58 (provides a premium, interactive web interface for file uploads, live diagnostics, and history tracking).
- **Serialization:** Joblib (used to save the trained Random Forest Classifier, StandardScaler, and column indexes, avoiding retraining on startup).
- **Inference & Features:** Scikit-learn (pre-trained modeling), Pandas & NumPy (data transformations and alignment).
- **Computer Vision & Quality Audits:** OpenCV 4 (performs real-time calculations of image brightness and Laplacian variance for focus).

2. Modular Project Structure

To make the code maintainable and deployment-ready, we restructured our Jupyter notebook code into separate modules:

- **data/:** Houses `plantvillage\_metadata.csv` representing dataset metrics.
- **models/:** Contains serialized `.joblib` objects for model and scaler weights.
- **src/preprocessing.py:** Handles data cleaning and real-time image quality validation filters.
- **src/features.py:** Implements feature engineering (resolution, aspect ratio, density, and brightness-blur interactions).
- **src/predict.py:** Handles model loading, input alignment, feature scaling, and inference outputs.
- **app.py:** The main Streamlit web application running the UI layout.

3. Technical Challenges & Resolutions

Technical Issue Faced	Impact on Project	Resolution Implemented
Bridging Image Input with Tabular Models	The Random Forest classifier expects tabular quality inputs, but users upload raw images (JPEG/PNG).	Implemented dynamic real-time feature extraction in <i>app.py</i> . Upon file upload, OpenCV computes the average gray value (exposure) and Laplacian variance (focus) instantly, mapping them directly to model inputs.
Mocking Grad-CAM for Non-Deep Models	Tabular algorithms (Random Forest) do not natively produce Grad-CAM activation maps to display affected areas.	Created a custom HSV mask filter in <i>app.py</i> that isolates brown/yellow necrotic spot color ranges. We apply a Gaussian blur to create a heatmap, blending it back with transparency to visual match Grad-CAM overlays.
Offline Environment & Testing Constraints	Sandbox has zero internet access, causing remote sample leaf image downloads to fail during setup.	Designed a custom drawing script utilizing PIL. Upon first launch, the backend programmatically draws a leaf vector (polygon leaf with veins and brown disease spots), generating the local testing image automatically.
Data Alignment on Inference	Dummies generated by One-Hot encoding of crop types during inference can cause columns to mismatch.	We serialized the expected feature layout list as <i>feature_names.joblib</i> . On prediction, we reindex the input dataframe to align with this template, automatically filling missing categories with 0.

4. Verification Results

The application structure was verified by executing the Streamlit instance locally. Models loaded successfully from serialized binaries, and real-time validation flags (blur/exposure) operated correctly, providing instant classifications, confidence probabilities, and treatment advisories based on the underlying metadata model.